Clients Rot, Connections Don't: Degradation, Silent Failures and Refresh in Long-Lived WMI Connections

Author Name
Affiliation

Abstract

Windows management agents commonly recycle their WMI connections every few hours on the belief that long-lived connections degrade. We test that belief with a client that is correct by construction, checks results against non-WMI ground truth, and attributes resource use to the provider host serving each query. On Windows Server 2019, one set of connections served 381,600 queries over 6 h at 100× a production agent's rate (about 25 production-days) with no measurable growth in client memory (0.037 MB/h, 95% CI [−0.007, 0.082]), handles, provider-host memory or latency. The degradation operators observe is client-induced: an unreleased `VARIANT` array leaks about 140–300 B per read, and cross-apartment calls cost about 3× the CPU per query. *Silent* failures matter more: a popular Go COM library ends enumeration without reporting the error, three of a production agent's queries have failed on every poll while logged as successes, and `Win32_SystemDriver` omits a driver depending on the properties selected. Fault injection shows where refresh matters: a restart of the WMI service breaks every existing connection permanently; reconnecting on the error restored service within 6 s, a 1-hour timer after 30 min, and a replica of Agent A's pool not within the hour, because its health check does not recognise the error. Agents should invest in client correctness and result verification, and reconnect on errors rather than on a clock.

1 Introduction

Windows Management Instrumentation (WMI) is the standard inventory and telemetry interface on Windows; endpoint agents, monitoring systems and configuration tools poll it continuously. A widespread practice is to treat WMI connections as perishable: agents tear down and recreate their `IWbemServices` connections on a timer or a query budget. The production agent we studied ("Agent A") recycles every 6 h or 50,000 queries, a rule motivated by observed memory growth, latency creep and timeouts.

Whether those symptoms come from *connection age* or from the *client* has, to our knowledge, not been measured. It is hard to measure: an incorrect client produces exactly these symptoms, the usual metric (memory summed over all `WmiPrvSE.exe` processes) mixes every WMI client on the host, and many WMI failures are silent.

We contribute (1) a method that removes these confounders: a self-tested correct client, per-provider-host attribution, an ETW meter of foreign WMI clients, and non-WMI ground truth (§3); (2) evidence that a correct client's connections do not degrade at accelerated query counts (§4); (3) measurements of two client bugs that produce the folklore symptoms, and a catalogue of silent failures (§5, §6); and (4) a fault-injection comparison of refresh strategies with concrete guidance (§7, §8).

2 Background

A client calls `IWbemServices` in the WMI service (`Winmgmt`), which dispatches each class to a provider. In-process providers run in shared `WmiPrvSE.exe` host processes, one per hosting model such as NetworkService or LocalService [1]. WMI unloads a provider once it has been idle for its cache-control interval (`ClearAfter`; unloading takes "anywhere from the interval … to twice that interval") [2, 3], and enforces per-host quotas on memory, threads and handles [4]. Queries issued with the return-immediately and forward-only flags (0x30) are semisynchronous: `ExecQuery` returns an enumerator at once, and errors surface later from `IEnumWbemClassObject::Next`, which returns `WBEM_S_FALSE` at the normal end of a result [5]. The scripting objects are apartment-threaded: a call from another apartment is marshalled to the owning single-threaded apartment (STA) [6], and property values arrive as `VARIANT`s that the caller must free with `VariantClear`, which frees strings and arrays [7].

3 Method

A correct client. The probe (Go, `go-ole`) gives every connection its own OS thread initialised as an STA, and makes every COM call for that connection, including release, on that thread. Every `VARIANT` is cleared exactly once; a ledger counts acquisitions and releases per query and flags outstanding or double releases. Enumeration uses a strict loop: `S_OK` yields an item, `S_FALSE` with no item ends the result, and any other code is an error recorded with its `HRESULT`, WBEM code and the operation that failed. A query that exceeds its deadline is abandoned on the caller's side and its row is written when WMI eventually answers.

Workload. The catalogue reproduces Agent A's 23 production queries verbatim (processes, services, drivers, event logs, firewall, Defender, WMI subscriptions, scheduled tasks, Device Guard, TPM), each at its production rate; 22 queries in 7 namespaces exist on our host (BitLocker's is absent), 636 queries/h in total. An acceleration factor k multiplies every rate. A pre-flight calibration on the test host (10-minute steps, $k \in \{1, 5, 10, 20, 50, 100\}$) gave probe-induced CPU (probe + all provider hosts + Winmgmt) of 0.4, 1.6, 3.3, 6.3, 15.6 and 29.9% with no dropped queries; we use $k_H = 100$, the top of the ramp, below a 40% budget.

Attribution and isolation. Every 10 s the probe samples itself, the `Winmgmt` service host (PID from the service control manager) and each `WmiPrvSE` by PID and creation time, labelled with its account and the provider DLLs it has loaded; this maps each queried class to the host serving it. Nothing here uses WMI. A real-time ETW session on `Microsoft-Windows-WMI-Activity` [8] counts operation starts by client PID, so foreign WMI clients are measured, not assumed absent. We found that `tasklist.exe`, the CIM cmdlets and PowerShell remoting all issue WMI calls; host inspection during runs was restricted to tools measured at zero WMI operations.

Ground truth. Results are compared with non-WMI sources taken immediately before and after each query: the process list (`NtQuerySystemInformation`), the service control manager, and invariants captured at the start. Five silent classes are counted: S1 impossible-empty, S2 truncated (a key present in both truth snapshots is missing), S3 enumeration ended by an error (what a lenient loop would report as success), S4 partial objects, S5 content mismatch.

Testbed and statistics. Windows Server 2019 Standard (build 17763.3650), 2 vCPU, 4 GB, Defender real-time protection on, no other WMI-heavy software. Growth is the OLS slope of 10 s samples after a 30-minute warm-up with Newey–West standard errors (15-minute lag window) [10]. Pass criteria were fixed before the runs: client private-byte slope < 1 MB/h and < 3 B per array read (upper 95% bound), handle range ≤ 50, and ≤ 5 MB retained after a 1-hour release test.

4 A correct client does not degrade

The self-test arm doubles as our longest correct-client run (Table 1). At $k = 100$, its 6 hours carry the query count of about 25 days at the production rate. All client invariants held on every query (correct apartment, no outstanding, double or leaked `VARIANT`s, no panics). Median latency per query key stayed flat across connection age, from about 1 ms (repository classes) to 250 ms (`MSFT_NetFirewallRule`, 260 rows), and neither the `NetworkServiceHost` provider host serving 6 providers nor the `Winmgmt` host grew (Fig. 1). The single handle step (339 to 347) coincides with one additional Go runtime thread and is flat before and after. A 7-day run at $k = 100$ (about 300 production-days of queries per connection) is in progress and will replace this section's age bound.

Table 1: ST-1: one set of connections, strategy *none*, $k = 100$, 6 h (Server 2019). CIs are 95%, HAC.

Queries	381,600
ok / deterministic error / abandoned	360,000 / 21,600 / 0
Silent failures above truth baseline	0
Client private bytes, slope (MB/h)	0.037 [−0.007, 0.082]
per array read (B; 3.48 M reads)	0.06 [−0.01, 0.14]
Client handles: range	17
slope per 1k queries	0.034 [0.029, 0.039]
Retained after 1 h release test	−0.39 MB
Provider hosts, Winmgmt host	flat (Fig. 1)
Recycles; induced CPU	0; 30.2%

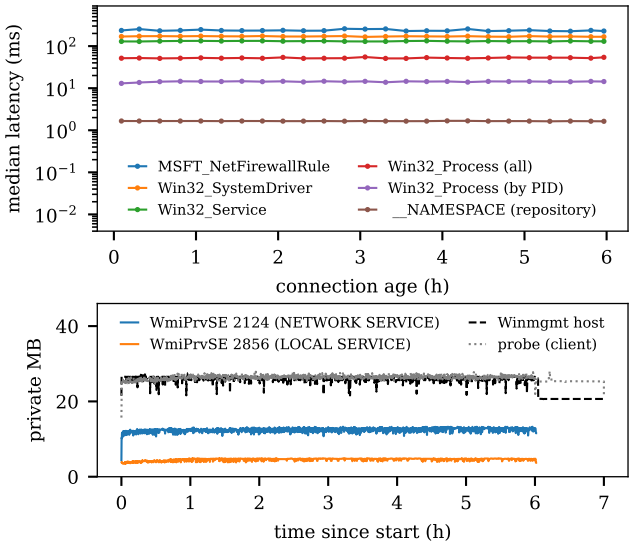


Figure 1: ST-1. Top: median latency per 15-minute window against connection age for six query keys. Bottom: private bytes of the two provider hosts (labelled by account and provider DLLs) and of the Winmgmt host; connections were released at 6 h.

Providers reload anyway. On this host `ClearAfter` is 30 s (15 s for one class). At the production rate ($k = 1$) most classes are queried less often, and the WMI-Activity log showed six providers reloading every 1–2 minutes *inside* one long-lived host PID (event 5857 within 10 minutes); the Defender provider's host itself was replaced, and idle hosts exited 60–65 s after release. A production client thus gets a free refresh of provider state every few minutes; only its connection and the WMI service age.

5 What degrades is the client

Unreleased arrays (Agent A, issue 115). Property values read through `Properties_` that were never passed to `VariantClear` leak their `SAFEARRAY`s. In production this was steady memory growth (+1.45 MB/h over 13.5 h on a Win-

dows 10 VM, +2.29 MB/h over 20 h on a laptop) that scaled with the firewall collector's frequency ($+8.8 \pm 0.6$ MB/h at 60 s, $+0.44 \pm 0.52$ after the fix) and that *recycling did not cure* (2.5 MB/h with recycling working): the leak is in the client. Our positive control re-enables the bug in the probe: at $k = 10$ for 1 h (3,180 queries and 31,650 array reads in the 30-minute measurement window) the client's private bytes grew 8.5 MB/h, i.e. 141.5 B per array read (95% CI [139.3, 143.6]), against 0.06 B for the correct client (Table 1); the probe's own ledger flagged 780 unreleased values. The leak grows with reads, not with connection age, and Agent A's data above show that recycling does not release it.

Cross-apartment calls (issue 118). Agent A called the apartment-threaded scripting objects from multi-threaded-apartment goroutines, so COM marshalled every call into a single host STA (confirmed in debugger stacks). Moving each connection to its own STA cut CPU per collection cycle about 3× (process poll 1.31–1.40 s to 0.42–0.44 s; firewall rules 8.5 s to 2.8 s) and steady-state CPU on a busy 16-CPU laptop from 28.3% to 3.5–3.9% of a core, eliminating 1,490 abandoned calls in 69 h (≈21/h). On a quiet 2-vCPU VM the same bug cost 5.0% vs 0.9% and caused no abandoned calls: the symptom scales with host load, which makes it look like ageing on busy machines.

6 Silent failures

A library that swallows errors. In `go-ole` v1.3.0 [9], `oleutil.ForEach` (`oleutil.go`, lines 118–126) loops while the number of items returned is positive:

```
118  for item, length, err := enum.Next(1); length > 0;
119      item, length, err = enum.Next(1) {
120    if err != nil { return err }
121    ...
122  }
123  return nil
```

When `Next` fails with zero items, the loop condition is false, the error is never inspected and the function returns `nil`. On Server 2019, `SELECT * FROM` a non-existent class fails only at `Next` with `WBEM_E_INVALID_CLASS` (0x80041010); `ForEach` reports success with 0 items, while our strict loop records the error.

Production queries that never worked. Three of Agent A's 23 queries fail at `Next` on every call (Table 2). Before the agent was stopped, WMI-Activity recorded 84 `5858` errors for each in 7 h, while the agent's audit log shows a 0-row, 0 ms success at each of their positions in every cycle (the log has no query text, so attribution is by position). From the agent's code, the consequences are concrete: firewall rules are shipped without their application path, non-standard event consumers are missing from the persistence inventory, and scheduled tasks lack last-run and next-run times. Each was a plausible-looking query: `Name` exists only on derived consumer classes,

Table 2: Silent failures observed on Server 2019. All return success to a lenient client.

Query / class	Observed
`SELECT Name FROM __EventConsumer`	`0x80041017` at `Next`, every call
`SELECT InstanceID, Program, Package FROM MSFT_NetApplicationFilter`	`0x80041017` at `Next`, every call
`... FROM MSFT_ScheduledTaskInfo`	`0x80041010` at `Next`, every call
`Win32_SystemDriver` with any configuration property (or `*`)	333 rows; `vwifibus` omitted (binary absent); `SELECT Name` returns 334

`MSFT_NetApplicationFilter` exposes `AppPath` rather than `Program`, and `MSFT_ScheduledTaskInfo` is not a WMI class on this OS.

A provider that drops rows. `Win32_SystemDriver` returned 333 drivers whenever the query selected any configuration property (`DisplayName`, `PathName`, `StartMode`, `StartName`, `ServiceType`, `Description`, or `*`), and 334 for `SELECT Name` or `SELECT Name, State`. The missing driver, `vwifibus`, is registered with the service control manager but its binary is absent. No error is returned at any layer; only the comparison with the service control manager reveals it. Because the omission is deterministic, we treat it as a per-key truth baseline and count S2 only above it.

Races are not failures. In 19,716 calibration queries the only other flags were three scheduled-task counts one short as the count changed and one process that exited during its query: timing, not WMI, separable only by bracketing truth before and after each query.

7 Fault recovery and refresh

If connections do not age, refresh is only worth its cost as recovery from faults. We ran four 90-minute arms back to back ($k = 100$, fresh connections each), one per strategy: *none* (E0); *timed*, redial every connection after 1 h (E1); *adaptive*, retire a connection when a failed call's effective error code is connection-level and redial at its next query (E2); and a replica of Agent A's *pool*, which recycles at 6 h, 50,000 queries or 5 min idle, or when the outer `HRESULT` maps to access denied, transport failure, "not available" or COM not initialised (E3; rules copied from the agent's source). At 15 min we killed the `WmiPrvSE` host serving `Win32_Process` and five other providers; at 30 min we restarted `Winmgmt` and its two running dependent services through the service control manager (1.9 s in total). The three deterministically failing keys are excluded.

Table 3: E-fault, $k = 100$, 90 min per arm (Server 2019). Errors in the 15 min after the host kill (15,000 queries) and the 60 min after the Winmgmt restart (60,000 queries); recovery is the time until every connection that failed has answered again.

		Kill	Winmgmt restart	
Arm	Strategy	errors	errors	recovery
E0	none	0	59,995	none in 60 min
E1	timed (1 h)	0	30,002	1,804.5 s
E2	adaptive	0	8	6.0 s
E3	Agent A pool	0	59,995	none in 60 min

A provider-host crash is invisible. In all four arms, killing the provider host caused no failed query: the connection is to `Winmgmt`, which had a replacement host with the same providers running 31 ms later (E0), and the 99th-percentile latency in the following 30 s was unchanged (225 vs. 231 ms before, E0).

A service restart is permanent. After `Winmgmt` restarted, every call on every existing connection failed with `RPC_S_SERVER_UNAVAILABLE` (0x800706BA) and none recovered on its own (E0: 59,995 of 60,000; the 5 successes were calls issued in the 175 ms before the service stopped). The timed strategy was down until its timer fired, 30 minutes later; an outage of up to one refresh interval is inherent to it. The adaptive client lost exactly one query per connection: each of the 9 connections failed once, was retired, redialled in 1–6 ms, and its next scheduled query succeeded; the 6.0 s is the query interval of the least-busy namespace at $k = 100$ and would be one production polling interval at $k = 1$. Agent A's pool behaved exactly like no refresh (59,995 errors, no recycle): `RPC_S_SERVER_UNAVAILABLE` is not an outer code its health check treats as unhealthy, and at arm end its connections were 1.5 h of 6 h old with at most 21,000 of 50,000 queries. By its rules the first recycle would come from the query budget at about 214 min ($k = 100$) or the 6 h age limit (production rate); we did not run that long. The refresh mechanism did not fire on the one fault that needed it.

8 Discussion

Guidance. (1) Make the client correct first: one STA per connection, all COM calls on it, every `VARIANT` cleared, and a test that runs a few hundred thousand queries and checks for growth; timed recycling hides neither of the bugs above. (2) Never let an enumeration end without inspecting the last `HRESULT`; audit libraries for this pattern. (3) Validate each query once at start-up and verify results against an independent source where one exists; "0 rows" is a claim, not a fact. (4) Reconnect on connection-level errors, including the RPC codes a service restart produces (`RPC_S_SERVER_UNAVAILABLE`), `WBEM_E_TRANSPORT_FAILURE` and `WBEM_E_NOT_AVAILABLE`, by inspecting the effective error code, not only the outer `HRESULT`; test the health check by restarting `Winmgmt`. A timer adds churn and leaves an outage of up to one interval. (5) Do not attribute WmiPrvSE growth to your client without per-host attribution and a foreign-client meter.

Threats to validity. One host, one OS build, one workload; each fault was injected once per strategy. Accelerating the query rate tests count, not age: our longest correct-client connection is 6 h old; the 7-day run is in progress. The ETW meter ran one minute in fifteen during arms, so foreign activity is sampled (it saw one burst of 6 operations, most likely the Windows Update service). Agent A's production numbers come from before/after comparisons of single changes on a few hosts. Cross-OS replication (Windows 10, Server 2016/2022) is future work.

Related work. Microsoft documents the mechanisms we rely on: provider hosting and unloading [1, 2], quotas [4], enumeration semantics [5] and COM apartment rules [6]. We are not aware of published measurements of WMI connection ageing under a verified-correct client, nor of a catalogue of WMI silent failures; we welcome pointers.

9 Conclusion

Long-lived WMI connections, used correctly, did not degrade in our measurements; the symptoms attributed to ageing were produced by the client, and the costliest failures were silent. Refresh is a recovery mechanism, best driven by errors. The probe, catalogue and analysis code are available from the authors.

References

[1] Microsoft. *Provider Hosting and Security*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/provider-hosting-and-security

[2] Microsoft. *Unloading a Provider*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/unloading-a-provider

[3] Microsoft. *__CacheControl class*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/--cachecontrol

[4] Microsoft. *__ProviderHostQuotaConfiguration class*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/--providerhostquotaconfiguration

[5] Microsoft. *IEnumWbemClassObject::Next*. https://learn.microsoft.com/en-us/windows/win32/api/wbemcli/nf-wbemcli-ienumwbemclassobject-next

[6] Microsoft. *Single-Threaded Apartments*. https://learn.microsoft.com/en-us/windows/win32/com/single-threaded-apartments

[7] Microsoft. *VariantClear function*. https://learn.microsoft.com/en-us/windows/win32/api/oleauto/nf-oleauto-variantclear

[8] Microsoft. *Tracing WMI Activity*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/tracing-wmi-activity

[9] go-ole contributors. *go-ole v1.3.0, oleutil/oleutil.go*. https://github.com/go-ole/go-ole/blob/v1.3.0/oleutil/oleutil.go

[10] W. K. Newey and K. D. West. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3):703–708, 1987.